Université de Haute-Alsace - ENSISA
Informatique et Réseaux 2A - 2ème semestre

Deep Learning 1 - Projet final
Architectures d'apprentissage profond pour la classification d'ECG

Auteurs :

DEBOUCHER Mathys
FRADIN Mahe
GARRONE Gabriel

Table des matières

1 Contexte et présentation des données **2**
1.1 Contexte applicatif 2
1.2 Jeu de données ECG200 2
1.3 Pré-traitement des données 2

2 Architectures des modèles **2**
2.1 Multi-Layer Perceptron (MLP) 3
2.2 Réseau de neurones convolutif 1D (CNN1D) . . . . . .
3
2.3 Réseau de neurones récurrent avec LSTM . . . . . .
4
3.1 Protocole commun 4
3.2 Régularisation et arrêt précoce . . . . . .
5
5
4.1 Métriques utilisées 5
4.2 Résultats numériques . . . . . .
6
4.3 Courbes d'apprentissage 6
4.4 Matrices de confusion 6
4.5 Comparaison visuelle des performances . . . . . .
7
5.1 Résultats de complexité 7
5.2 Analyse de la complexité 8

6 Analyse des résultats et discussion **8**
6.1 Le MLP : le meilleur, contre toute attente 8
6.2 Le CNN1D : un classifieur trivial 8
6.3 Le RNN_LSTM : trop complexe pour le jeu de données 9
6.4 Limites et pistes d'amélioration 9

7 Conclusion **10**

1 Contexte et présentation des données

1.1 Contexte applicatif

Dans ce rapport, nous présentons notre travail sur la classification automatique de signaux ECG (*Électrocardiogramme*) pour l'aide au diagnostic cardiaque. Les cardiologues analysent chaque jour un grand nombre de signaux, et une méthode automatique capable de repérer les infarctus du myocarde leur ferait gagner un temps précieux. Nous avons donc cherché à construire et comparer plusieurs architectures de Deep Learning pour distinguer un rythme normal d'un rythme anormal.

1.2 Jeu de données ECG200

Nous travaillons avec le jeu de données **ECG200** de l'archive UCR Time Series Classification. Il contient 100 séries temporelles pour l'entraînement et 100 pour le test, chacune composée de 96 points temporels représentant l'activité électrique d'un battement cardiaque. Les deux classes sont le rythme **normal** (classe 0, 36 exemples dans le test) et le rythme **anormal** correspondant à un infarctus (classe 1, 64 exemples dans le test).

On peut noter le **déséquilibre entre les classes**. Avec 64 % de cas positifs, un classifieur qui prédirait systématiquement "infarctus" atteindrait déjà 64 % d'exactitude sans rien apprendre. C'est pourquoi nous n'avons pas limité notre évaluation à cette seule métrique. Les fichiers sont au format `.txt` avec la première colonne correspondant au label.

1.3 Pré-traitement des données

Les données brutes sont normalisées par un **MinMaxScaler** (scikit-learn), ramenant chaque valeur dans l'intervalle $[0, 1]$. Nous avons choisi cette méthode pour plusieurs raisons :

- Les réseaux de neurones sont sensibles à l'échelle des entrées ; une normalisation accélère la convergence et stabilise l'optimisation par descente de gradient.
- Le MinMaxScaler préserve la forme des signaux, ce qui est important pour des données temporelles structurées.
- Le scaler est **ajusté uniquement sur les données d'entraînement** (`fit_transform`) et appliqué tel quel au jeu de test (`transform`), évitant ainsi toute fuite d'information (*data leakage*).

Pour le MLP, les séries sont des vecteurs de dimension 96. Pour le CNN et le RNN, elles sont reformatées en tenseurs de dimension $(T, 1)$.

2 Architectures des modèles

Nous avons implémenté trois architectures distinctes. Toutes partagent la même tête de classification (une couche `Dense` à 2 neurones avec `softmax`) et le même optimiseur **Adam** (taux d'apprentissage 10^{-3}) avec la perte `categorical_crossentropy`. La différenciation vient de la façon dont elles traitent la dimension temporelle du signal.

2.1 Multi-Layer Perceptron (MLP)

Architecture

$$\begin{aligned}&\text{Input}(96)\\&\quad\to \text{Dense}(128, \text{ReLU}) \to \text{Dropout}(0.3)\\&\quad\to \text{Dense}(64, \text{ReLU}) \to \text{Dense}(2, \text{Softmax})\end{aligned}$$

Justification des choix Le MLP est notre modèle de référence. Il ignore volontairement la structure temporelle et traite le signal comme un simple vecteur de 96 valeurs. C'est justement cette simplicité qui l'intéresse dans notre contexte car on a seulement 100 exemples d'entraînement et il vaut mieux éviter les architectures trop complexes.

Pour les deux couches cachées (128 puis 64 neurones), nous avons opté pour une structure en entonnoir. Chaque couche compresse progressivement l'information, forçant le réseau à ne conserver que l'essentiel. Ajouter une troisième couche n'apporterait probablement rien et aggraverait le sur-apprentissage.

L'activation **ReLU** est le choix standard pour les couches cachées, car elle évite la saturation du gradient qu'on observe avec sigmoid ou tanh.

Enfin, le **Dropout à 0.3** est notre principale technique de régularisation. En désactivant aléatoirement 30 % des neurones à chaque itération, il empêche le réseau de trop mémoriser les exemples d'entraînement. Ce taux nous a semblé raisonnable car il est assez élevé pour régulariser mais pas trop pour ne pas nuire à l'apprentissage.

Nombre de paramètres $96 \times 128 + 128 + 128 \times 64 + 64 + 64 \times 2 + 2 = \mathbf{20\,802}$

2.2 Réseau de neurones convolutif 1D (CNN1D)

Architecture

$$\begin{aligned}&\text{Input}(96, 1)\\&\quad\to \text{Conv1D}(32,\ k{=}5,\ \text{same},\ \text{ReLU}) \to \text{MaxPool1D}(2)\\&\quad\to \text{Conv1D}(64,\ k{=}5,\ \text{same},\ \text{ReLU}) \to \text{GlobalAvgPool1D}\\&\quad\to \text{Dense}(64, \text{ReLU}) \to \text{Dense}(2, \text{Softmax})\end{aligned}$$

Justification des choix L'idée derrière un CNN 1D pour de l'ECG est assez naturelle. Les ondes P, QRS et T sont des structures locales dans le signal, et les filtres convolutifs ont été créés pour détecter ce type de motifs, quelle que soit leur position dans la séquence.

Nous avons utilisé **deux blocs convolutifs** (32 puis 64 filtres) pour apprendre des motifs à différents niveaux d'abstraction, en doublant le nombre de filtres à chaque étage. Le **noyau de taille 5** représente un compromis. S'il est trop petit (1-2), il ne capture pas assez de contexte. S'il esttrop grand (>10), il risque de mélanger des structures distinctes. Le padding `same` conserve la longueur temporelle entre les couches, ce qui simplifie l'architecture.

Le **MaxPooling1D(2)** réduit la dimension temporelle d'un facteur 2 après le premier bloc, introduisant une légère invariance translationelle et allégeant le calcul des couches suivantes.

Le choix du **GlobalAveragePooling1D** en fin de réseau mérite une explication. Plutôt que d'aplatir toute la carte de features avec `Flatten` (ce qui produirait un grand vecteur

et beaucoup de paramètres supplémentaires), on calcule la moyenne de chaque canal sur la dimension temporelle. Cela donne une représentation compacte et naturellement régularisée de la séquence, comme décrit dans Lin et al. Sur un petit jeu de données comme ECG200, cette régularisation implicite est précieuse.

Nombre de paramètres **14 786**, le modèle le plus compact des trois.

2.3 Réseau de neurones récurrent avec LSTM

Architecture

$$\begin{aligned}&\text{Input}(96, 1)\\&\quad \to \text{LSTM}(64,\ \text{return_sequences=True}) \to \text{LSTM}(32)\\&\quad \to \text{Dense}(32, \text{ReLU}) \to \text{Dense}(2, \text{Softmax})\end{aligned}$$

Justification des choix Les RNN maintiennent un état caché qui se met à jour à chaque pas de temps, ce qui leur permet en théorie de mémoriser des dépendances sur l'ensemble de la séquence. Cela peut être intéressant pour un signal ECG car la relation entre l'onde P et l'onde T (par exemple) s'étend sur une grosse partie du battement.

Nous avons choisi des cellules **LSTM** plutôt qu'un RNN simple, car les RNN classiques ont un problème de gradient évanescent. Plus la séquence est longue, plus les informations du début sont difficiles à transmettre. Les LSTM règlent ce problème avec leurs portes (oubli, entrée, sortie) qui contrôlent ce que la cellule retient ou oublie. Avec 96 pas de temps, ce mécanisme nous a semblé justifié.

Nous avons empilé **deux couches LSTM** (64 puis 32 unités). La première retourne toute la séquence d'états cachés (`return_sequences=True`) pour que la deuxième puisse les traiter, ce qui permet d'apprendre des abstractions temporelles à deux niveaux. La réduction $64 \to 32$ compresse progressivement la représentation. La couche **Dense(32, ReLU)** avant la sortie ajoute une transformation non linéaire supplémentaire avant la décision finale.

Nombre de paramètres **30 434**, le modèle le plus paramétrique des trois en raison des matrices de poids des cellules LSTM.

3 Méthodologie d'entraînement

3.1 Protocole commun

Pour que la comparaison soit équitable, nous avons appliqué exactement le même protocole d'entraînement aux trois modèles.

Nous avons utilisé l'optimiseur **Adam** ($\eta = 10^{-3}$) qui permet d'adapter le taux d'apprentissage de chaque paramètre indépendamment, en combinant les avantages de RMSprop et du momentum. La perte `categorical_crossentropy` est adaptée à notre encodage one-hot des labels.

Concernant la **taille de batch**, nous avons choisi 16. Avec seulement 80 exemples effectifs pour l'entraînement (20 % mis de côté pour la validation), un batch plus grand réduirait trop le nombre de mises à jour par époque. Un batch petit introduit du bruit dans le gradient, ce qui peut jouer un rôle de régularisation implicite.

Le nombre maximal d'époques est fixé à 150, mais dans la pratique l'arrêt précoce intervient bien avant. Enfin, une **graine aléatoire fixe** (`seed=42`) est appliquée à TensorFlow et NumPy pour que les résultats soient reproductibles.

3.2 Régularisation et arrêt précoce

Early Stopping C'est probablement la décision de régularisation la plus importante du projet. Nous surveillons la perte de validation avec une patience de 20 époques. Si elle ne s'améliore pas pendant 20 époques consécutives, l'entraînement s'arrête. L'option `restore_best_weights=True` est essentielle car elle revient au meilleur état du modèle, et non à l'état final qui peut être sur-ajusté. Sans cela, l'Early Stopping n'aurait que peu d'intérêt.

ModelCheckpoint Un checkpoint est sauvegardé à chaque amélioration de la validation, ce qui permet de récupérer le meilleur modèle même si l'entraînement est interrompu.

Dropout (MLP uniquement) Seul le MLP intègre une couche Dropout. Pour le CNN et le LSTM, nous avons délibérément évité d'en ajouter car ces architectures sont plus complexes et l'Early Stopping assure une régularisation suffisante. Empiler trop de mécanismes de régularisation sur un si petit jeu de données risquait de nuire à l'apprentissage plutôt que de l'aider.

4 Évaluation des modèles

4.1 Métriques utilisées

Comme mentionné plus tôt, l'exactitude seule ne suffit pas ici. Nous avons donc utilisé un ensemble de métriques complémentaires, en gardant à l'esprit que dans un contexte médical, les deux types d'erreurs n'ont pas le même coût.

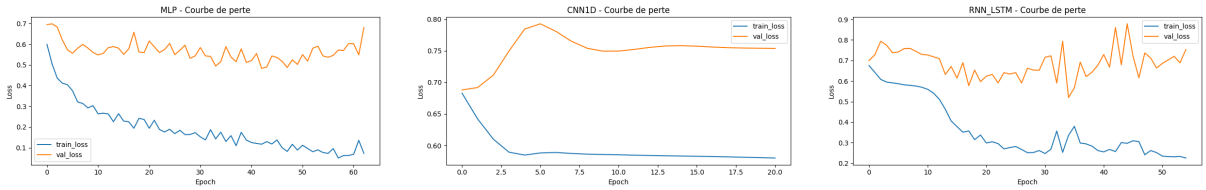
- **Exactitude (Accuracy) :** le pourcentage global de prédictions correctes. Facile à interpréter, mais trompeuse sur des classes déséquilibrées.
- **Précision :** parmi tous les cas détectés comme infarctus, combien le sont vraiment ? Une précision faible signifie beaucoup de fausses alarmes.
- **Rappel (Sensibilité)** parmi tous les vrais infarctus, combien sont détectés ? C'est la métrique la plus critique ici. Manquer un infarctus peut être fatal, alors qu'une fausse alarme est simplement inconfortable.
- **Score F1 :** moyenne harmonique précision/rappel. Utile pour comparer les modèles en un seul chiffre quand les deux métriques comptent.
- **Matrice de confusion :** elle permet de voir exactement où chaque modèle se trompe, ce qui s'avère particulièrement révélateur dans notre cas.

4.2 Résultats numériques

TABLE 1 – Résultats de performance sur l'ensemble de test (100 exemples)

Modèle	Accuracy	Précision	Rappel	F1	Paramètres
MLP	**0.85**	**0.866**	**0.906**	**0.885**	20 802
RNN_LSTM	0.74	0.788	0.813	0.800	30 434
CNN1D	0.64	0.640	1.000	0.780	14 786

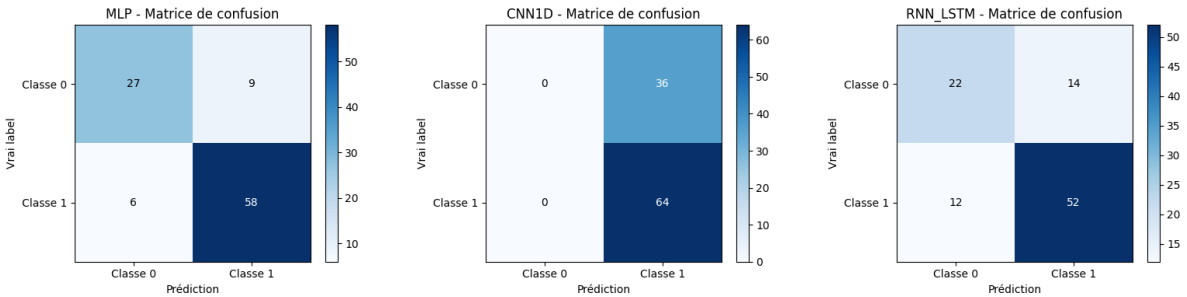
4.3 Courbes d'apprentissage



(a) MLP (b) CNN1D (c) RNN_LSTM

FIGURE 1 – Courbes de perte (entraînement vs validation) pour les trois modèles

4.4 Matrices de confusion



(a) MLP (b) CNN1D (c) RNN_LSTM

FIGURE 2 – Matrices de confusion sur l'ensemble de test. Classe 0 = normal, Classe 1 = infarctus.

Les matrices de confusion sont particulièrement parlantes et révèlent des comportements très différents entre les trois modèles.

Le **MLP** (TN=27, FP=9, FN=6, TP=58) est le seul à vraiment équilibrer les deux types d'erreurs avec 9 fausses alarmes et 6 infarctus manqués seulement, ce qui reste raisonnable dans les deux cas.

Le **CNN1D** (TN=0, FP=36, FN=0, TP=64) présente un cas extrême. Il prédit "infarctus" pour chaque exemple sans exception. Son rappel de 1.0 est donc totalement artificiel. Il ne rate aucun cas positif simplement parce qu'il ne prédit jamais la classe normale. Les 36 fausses alarmes sur 36 exemples normaux confirment qu'il s'agit d'un classifieur trivial.

Le **RNN_LSTM** (TN=22, FP=14, FN=12, TP=52) arrive à distinguer les deux classes, mais avec un taux d'erreur plus élevé que le MLP dans les deux sens, soit 14 fausses alarmes et 12 infarctus manqués.

4.5 Comparaison visuelle des performances

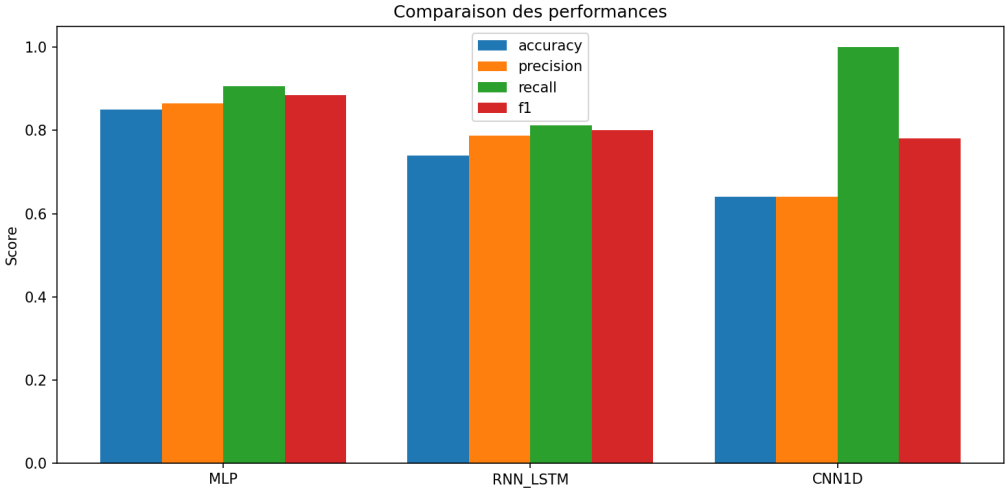


FIGURE 3 – Comparaison des métriques de performance pour les trois modèles

5 Étude comparative de complexité

5.1 Résultats de complexité

TABLE 2 – Comparaison de la complexité des modèles

Modèle	Paramètres	Taille (MB)	T. entr. (s)	T. inf. (s)	Époques
MLP	20 802	0.079	5.06	0.097	∼63
CNN1D	**14 786**	**0.056**	**2.62**	**0.128**	∼20
RNN_LSTM	30 434	0.116	16.28	0.460	∼55

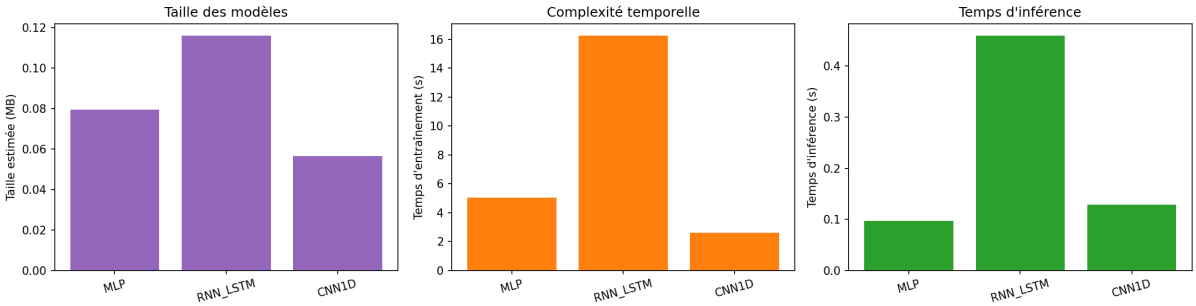


FIGURE 4 – Comparaison de la taille, du temps d'entraînement et du temps d'inférence

5.2 Analyse de la complexité

Taille des modèles Sans surprise, le CNN1D est le plus compact avec seulement 14 786 paramètres (0.056 MB). C'est l'effet du partage de paramètres propre aux convolutions, les mêmes filtres étant appliqués sur toute la séquence. Le MLP suit avec 20 802 paramètres (0.079 MB), et le LSTM est le plus lourd avec 30 434 paramètres (0.116 MB), car chaque cellule maintient des matrices de poids pour ses quatre portes. Cela dit, tous les modèles restent largement sous les 0.12 MB, ce qui convient parfaitement à un déploiement sur des machines modestes.

Temps d'entraînement Le CNN1D s'entraîne en à peine 2.62 s, suivi du MLP (5.06 s) et du LSTM (16.28 s). L'écart entre le CNN et le LSTM est frappant, le LSTM étant environ 6 fois plus lent. Cela vient du fait que le calcul récurrent est intrinsèquement séquentiel (chaque état caché dépend du précédent), ce qui limite fortement la parallélisation, contrairement aux convolutions et aux multiplications matricielles du MLP.

Temps d'inférence Sur 100 exemples de test, le MLP répond en 0.097 s, le CNN1D en 0.128 s, et le LSTM en 0.460 s. En pratique, toutes ces latences sont acceptables pour une analyse de routine, mais le LSTM est près de 5 fois plus lent que le MLP, ce qui pourrait devenir un frein si le volume de signaux à analyser est élevé.

Bilan complexité Sur le papier, le CNN1D domine sur tous les critères de complexité. Mais comme on le verra dans la section suivante, ses performances de classification sont très décevantes. C'est finalement le MLP qui offre le meilleur compromis global, performant, léger et rapide.

6 Analyse des résultats et discussion

6.1 Le MLP : le meilleur, contre toute attente

Avec 85 % d'exactitude et un F1 de 88.5 %, le MLP est de loin le meilleur modèle, ce qui est contre-intuitif puisque c'est l'architecture la plus simple et celle qui ignore le plus la structure temporelle du signal.

Sa courbe d'apprentissage (Figure 1a) montre un sur-apprentissage modéré. La perte d'entraînement descend jusqu'à environ 0.1 tandis que la perte de validation stagne autour de 0.5. Cela dit, grâce à `restore_best_weights`, le modèle sélectionné est celui du meilleur moment de validation, et non l'état final le plus sur-ajusté. Le Dropout a clairement aidé à contenir l'écart.

Pourquoi le MLP surpasse-t-il les deux autres ? Probablement parce que le problème reste de faible dimensionnalité (96 valeurs, 100 exemples). Dans ce régime, les motifs discriminants entre un ECG normal et un ECG ischémique sont suffisamment stables pour être capturés par un réseau entièrement connecté simple. Les architectures plus sophistiquées ajoutent de la complexité là où elle n'est pas nécessaire, et le jeu de données n'est pas assez grand pour en tirer profit.

6.2 Le CNN1D : un classifieur trivial

Le résultat du CNN1D est décevant. Sa matrice de confusion (Figure 2b) ne laisse aucun doute. Il prédit "infarctus" pour chaque exemple du jeu de test, sans exception. Son rappel de 1.0 est donc complètement trompeur. Il ne reflète pas une bonne détection, mais

simplement le fait qu'il ne prédit jamais la classe normale. Sa précision de 64 % correspond exactement à la proportion d'infarctus dans le test.

La courbe d'apprentissage (Figure 1b) confirme ce blocage. La perte d'entraînement stagne autour de 0.58-0.59, et la perte de validation ne bouge pratiquement pas après les premières époques. Le réseau est coincé dans un minimum local trivial dès le début.

À notre avis, deux facteurs principaux expliquent cet échec. D'abord, le jeu de données est tout simplement trop petit pour entraîner un CNN from scratch. Avec 80 exemples effectifs, les filtres convolutifs n'ont pas assez d'exemples variés pour apprendre des motifs vraiment discriminants. Ensuite, le déséquilibre de classes (64 % d'infarctus) pousse naturellement le réseau vers la classe majoritaire, et sans pondération des classes (`class_weight`), rien ne compense ce biais. Le MLP surmonte ce problème grâce à son Dropout et à sa moindre complexité ; le CNN n'a pas cet avantage ici.

6.3 Le RNN_LSTM : trop complexe pour le jeu de données

Le LSTM obtient 74 % d'exactitude et un F1 de 80 %, ce qui est correct mais inférieur au MLP. Sa courbe d'apprentissage (Figure 1c) raconte clairement son problème. La perte d'entraînement descend jusqu'à 0.23, tandis que la perte de validation oscille violemment entre 0.6 et 0.9. C'est un sur-apprentissage sévère. Avec 30 434 paramètres pour 80 exemples, le ratio est clairement défavorable.

On peut aussi questionner si le LSTM est vraiment adapté à ce problème dans ces conditions. Son atout principal (la mémoire à long terme) est surtout utile pour des séquences longues avec des dépendances distantes. Ici, les 96 pas de temps d'un battement cardiaque représentent un contexte relativement court, et les motifs discriminants (la forme de l'onde ST notamment) sont probablement locaux. Le MLP, en ignorant complètement la structure temporelle, capture ces différences avec moins de paramètres et moins de sur-apprentissage.

6.4 Limites et pistes d'amélioration

Notre principale limite méthodologique est l'absence de **validation croisée**. Avec seulement 100 exemples de test, les chiffres obtenus peuvent varier significativement d'une exécution à l'autre en fonction du partage entraînement/validation. Une validation k-fold ($k = 5$ ou $k = 10$) sur l'ensemble des 200 exemples aurait donné des estimations plus robustes et des intervalles de confiance, ce qui est particulièrement important dans un cadre médical où on veut être sûr que les performances sont stables.

Pour améliorer les modèles, plusieurs pistes nous semblent prioritaires :

- **Pondération des classes** pour le CNN : en pénalisant davantage les erreurs sur la classe minoritaire, on éviterait le biais vers la classe 1.
- **Augmentation de données** : ajouter du bruit gaussien, décaler légèrement les séries temporellement ou les redimensionner légèrement permettrait de multiplier artificiellement les exemples disponibles.
- **Dropout dans le CNN et le LSTM** pour mieux contenir le sur-apprentissage.
- Une **recherche d'hyperparamètres** plus systématique (architecture, taux d'apprentissage, taille de batch) apporterait probablement des gains supplémentaires.

7 Conclusion

Ce projet nous a amenés à construire et comparer trois architectures de Deep Learning pour la classification d'ECG. Les résultats sont instructifs, même si certains nous ont surpris.

Le **MLP** ressort largement en tête avec 85 % d'exactitude et un F1 de 88.5 %, loin devant le LSTM (74 %, F1 = 80 %) et le CNN1D qui s'est montré incapable de dépasser un comportement trivial. En termes de complexité, le CNN est le plus léger et le plus rapide à entraîner, mais ses mauvaises performances annulent cet avantage. Le LSTM est le plus coûteux sous tous les angles.

La leçon principale que l'on retient, c'est que la complexité d'un modèle ne garantit pas de meilleures performances, surtout quand les données sont rares. Sur un jeu d'entraînement de 100 exemples, un réseau simple et bien régularisé comme le MLP s'en sort mieux que des architectures conçues pour des problèmes plus complexes. Le Dropout et l'Early Stopping ont joué un rôle décisif pour éviter le sur-apprentissage.

Si l'objectif était de déployer une solution en clinique, nous recommanderions le MLP dans l'état actuel. Pour aller plus loin, les priorités seraient de disposer de plus de données, d'implémenter une validation croisée pour des estimations plus fiables, et d'ajouter une pondération des classes pour le CNN.

NB : Nous avons été assistés par Copilot avec l'auto-completion de code, ainsi que par Claude pour corriger la structure du rapport et améliorer la rédaction.